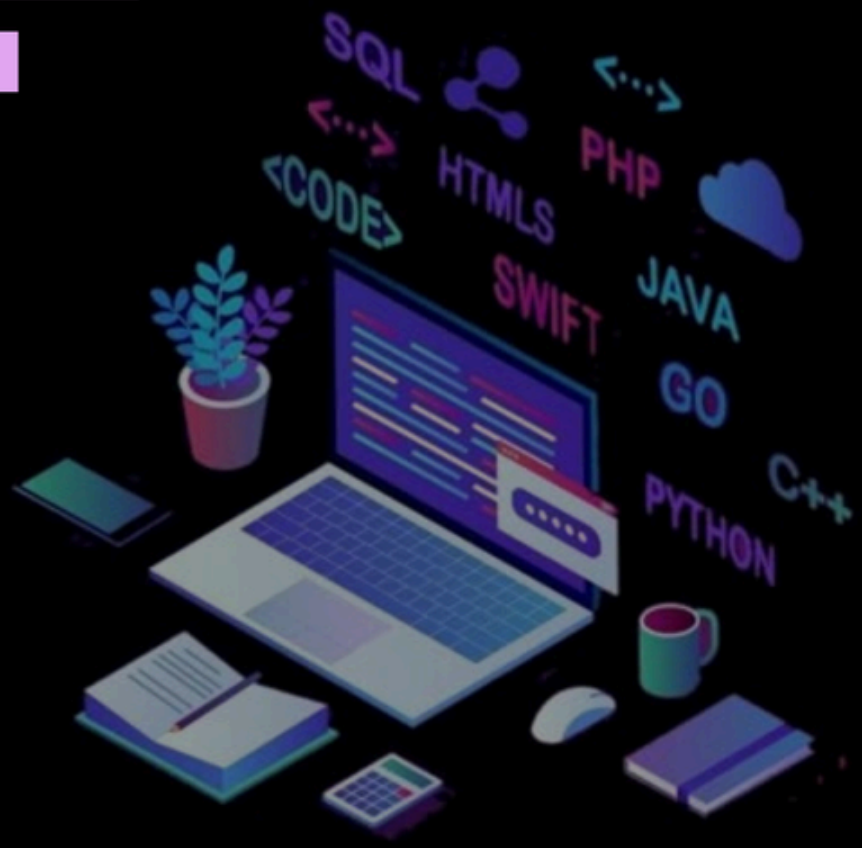
PROGRAMMERS CLUB

-DEPT. OF IT



The Programmers Club of the Information Technology Department is established to enhance the coding and problem-solving skills of students. It provides a platform to learn, collaborate, and innovate through coding quests, study materials, and practical solutions. The club aims to nurture technical excellence and prepare students for real-world challenges in the field of programming.

Faculty Co-ordinator :

Mrs. Pooja G. Dhawane

Student Co-ordinators:

Somesh Sharma

Om Kuthe

Nishant Lilhare

Tanmay Mankar

🧠 What is a Linked List?

A **Linked List** is a **linear data structure** where elements are stored in **nodes**.

Each node contains:

- **Data**
- **Pointer / Reference** to the next node

👉 Unlike arrays:

- Memory is **not contiguous**
- Size can **grow dynamically**
- Easy insertion and deletion

🔹 Example Structure

```
10 → 20 → 30 → 40 → NULL
```

Each element is called a **Node**

```
[ Data | Next ]
```

🔹 Node Representation

✅ C

```c
struct Node {
    int data;
    struct Node* next;
};
```

✅ C++

```cpp
class Node {
public:
    int data;
    Node* next;
};
```

✅ Java

```java
class Node {
    int data;
    Node next;
}
```

✅ Python

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

🔹 Basic Operations

✅ Insert at Beginning

Logic Steps

1. Create new node
2. Set newNode.next = head
3. Move head = newNode

✔ Example Code

C

```c
struct Node* insertBegin(struct Node* head, int x)
{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = head;
    head = newNode;
    return head;
}
```

C++

```cpp
Node* insertBegin(Node* head, int x)
{
    Node* newNode = new Node();
    newNode->data = x;
    newNode->next = head;
    return newNode;
}
```

Java

```java
Node insertBegin(Node head, int x)
{
    Node newNode = new Node();
    newNode.data = x;
    newNode.next = head;
    return newNode;
}
```

Python

```python
def insert_begin(head, x):
    newNode = Node(x)
    newNode.next = head
    return newNode
```

✅ Insert at End

Logic Steps

1. Create new node
2. Traverse till last node
3. Set last.next = newNode

✔ Example Code

C

```c
struct Node* insertEnd(struct Node* head, int x)
{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = NULL;

    if(head == NULL)
        return newNode;

    struct Node* temp = head;
    while(temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}
```

C++

```
Node* insertEnd(Node* head, int x)
{
    Node* newNode = new Node();
    newNode->data = x;
    newNode->next = NULL;

    if(head == NULL)
        return newNode;

    Node* temp = head;
    while(temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}
```

Java

```java
Node insertEnd(Node head, int x)
{
    Node newNode = new Node();
    newNode.data = x;
    newNode.next = null;

    if(head == null)
        return newNode;

    Node temp = head;
    while(temp.next != null)
        temp = temp.next;

    temp.next = newNode;
    return head;
}
```

Python

```python
def insert_end(head, x):
    newNode = Node(x)

    if head is None:
        return newNode

    temp = head
    while temp.next:
        temp = temp.next

    temp.next = newNode
    return head
```

✅ Traverse Linked List

Logic

```
current = head
while current ≠ NULL:
    print data
    move next
```

✔ Example Code

C

```c
void display(struct Node* head)
{
    struct Node* temp = head;
    while(temp ≠ NULL)
    {
        printf("%d ", temp→data);
        temp = temp→next;
    }
}
```

C++

```cpp
void display(Node* head)
{
    Node* temp = head;
    while(temp ≠ NULL)
    {
        cout << temp→data << " ";
        temp = temp→next;
    }
}
```

Java

```java
void display(Node head)
{
    Node temp = head;
    while(temp ≠ null)
    {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
}
```

Python

```python
def display(head):
    temp = head
    while temp:
        print(temp.data, end=" ")
        temp = temp.next
```

🔹 Reverse Linked List

Logic

Use **three pointers**:

- prev
- current
- next

Steps:

1. Store next node
2. Reverse link
3. Move forward

✔ Example Code (Common Logic)

C / C++

```
struct Node* reverse(struct Node* head)
{
    struct Node *prev = NULL, *curr = head, *next;

    while(curr != NULL)
    {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Java

```java
Node reverse(Node head)
{
    Node prev = null;
    Node curr = head;

    while(curr ≠ null)
    {
        Node next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Python

```python
def reverse(head):
    prev = None
    curr = head

    while curr:
        nextNode = curr.next
        curr.next = prev
        prev = curr
        curr = nextNode

    return prev
```

🔹 Find Middle Element

Technique → Two Pointer Method

- **Slow pointer** → moves 1 step
- **Fast pointer** → moves 2 steps

When fast reaches end → slow is middle.

✔ Example Code

C / C++

```
int findMiddle(struct Node* head)
{
    struct Node *slow = head, *fast = head;

    while(fast ≠ NULL && fast→next ≠ NULL)
    {
        slow = slow→next;
        fast = fast→next→next;
    }
    return slow→data;
}
```

Java

```java
int findMiddle(Node head)
{
    Node slow = head;
    Node fast = head;

    while(fast ≠ null && fast.next ≠ null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow.data;
}
```

Python

```python
def find_middle(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow.data
```

🔹 Time Complexity

Operation	Time
Insert Beginning	O(1)
Insert End	O(n)
Delete	O(n)
Search	O(n)

🔹 Common Mistakes

- ❌ Forgetting to update pointers
- ❌ Losing reference to head
- ❌ Not checking NULL conditions
- ❌ Creating infinite loops during traversal
- ❌ Incorrect reverse pointer order

🔹 Mini Practice (Dry Run)

Perform manually:

Insert:

```
5 → 10 → 15
```

Delete:

```
Remove 10
```

Result:

```
5 → 15
```

Reverse:

```
15 → 5
```

✨ **Stay Tuned For Upcoming Contests !!!**